

Adding Memory to LLM Conversations

Practice 2 Transcript | Oct 1, 2026 15:22

Sophia Martinez 00:00

Welcome! Our assistant works fine for single questions, but it forgets the conversation context too quickly. How do you think we should fix this?

You 01:44

We will resolve this by integrating Spring AI's `MessageChatMemoryAdvisor` with a distributed `ChatMemory` implementation backed by Redis, allowing the application to efficiently retain and inject conversation history into prompts while maintaining stateless scalability.

Sophia Martinez 01:47

That sounds like a solid approach! Redis is really great for distributed systems because of its speed and scalability. By using `MessageChatMemoryAdvisor`, you're essentially layering memory management into the system without burdening the core app logic, which is smart. How are you planning to handle the memory depth? Like, how much history will you store per user interaction?

You 02:24

Memory depth will be managed by configuring Spring AI's `MessageChatMemoryAdvisor` with a sliding window capped at the last 10 message pairs per session, combined with a Redis TTL policy to automatically purge inactive conversation history.

Sophia Martinez 02:27

Great call on the sliding window approach! Keeping it to the last 10 message pairs strikes a nice balance between maintaining relevant context and avoiding performance bottlenecks. Adding the Redis TTL policy is a really efficient way to manage stale data. Are you confident this setup will scale well as user interactions grow, or do you foresee any potential bottlenecks?

You 03:13

This Spring Boot AI architecture scales horizontally effectively, but potential bottlenecks such as Redis memory exhaustion under high concurrency and increased token processing latency must be mitigated by aggressively tuning TTL policies and implementing Spring AI's token-aware summarization advisors.

Sophia Martinez 03:15

Absolutely, those are key concerns to address. Tuning the TTL policies aggressively is a great way to prevent Redis from getting overwhelmed, and leveraging token-aware summarization advisors adds that extra layer of efficiency by condensing the context intelligently. Are you planning to monitor Redis metrics closely to fine-tune further as usage patterns evolve?

You 04:08

We will monitor system performance by leveraging Spring Boot Actuator and Micrometer to expose Redis cache metrics alongside Spring AI token consumption, enabling objective, data-driven adjustments to TTL policies and conversation memory configurations as usage patterns evolve.

Sophia Martinez 04:11

That's fantastic! Using Spring Boot Actuator and Micrometer for monitoring is a really proactive move—it'll give you clear insights into Redis and token consumption trends. With that data, adjusting TTL policies and memory configurations becomes super precise. Sounds like you're covering all bases! Anything else you're considering to further optimize this setup?

You 04:56

We will further optimize context management by integrating Spring AI's `VectorStore` and `RetrievalAugmentationAdvisor` to implement Retrieval-Augmented Generation (RAG), which semantically filters and injects only the most relevant historical interactions into the prompt to minimize token bloat.

Adding Memory to LLM Conversations

Sophia Martinez 04:59

Wow, that's a really smart addition! RAG is perfect for cutting down on token bloat while still keeping the context highly relevant. Combining `VectorStore`` with the `RetrievalAugmentationAdvisor`` gives you that semantic filtering edge. It's efficient and keeps things user-focused. With all these enhancements, your architecture is shaping up to be both scalable and user-friendly. Anything else you'd like to refine or dive deeper into?

You 08:52

We can further refine the architecture by exploring Spring AI's Model Context Protocol (MCP) for standardized enterprise tool integrations, or by configuring Micrometer to capture detailed observability metrics such as token consumption and API latency across the Retrieval-Augmented Generation pipeline.

Sophia Martinez 08:55

Those are excellent refinements! Leveraging Spring AI's Model Context Protocol for standardizing enterprise integrations could really streamline operations, making it easier to plug in additional tools as needed. And enhancing Micrometer observability to capture granular metrics like token usage and pipeline latency is a fantastic way to keep the system transparent and optimize continuously. You've got a really robust plan here—great job!

End of session
